

CATALOGGUARD LITE

상품 카탈로그 데이터 품질 검수 서비스

프로젝트 한 줄 소개

서로 다른 공급사 CSV 를 JSON 프로필로 표준화하고 상품의 누락·중복·가격·카테고리·개인정보 문제를 검수하여, 결과를 API·화면·CSV·PostgreSQL 이력으로 제공하는 Python·FastAPI 백엔드 프로젝트입니다.

프로젝트 요약

항목	내용
형태	개인 프로젝트 · 취업 포트폴리오용 MVP 고도화
담당	검수 규칙, FastAPI API, PostgreSQL 저장, Redis·Celery 비동기 처리, 공급사 ETL 프로필 2 종, 테스트·CI, 배포 검증
핵심 기술	Python, FastAPI, PostgreSQL, SQLAlchemy, Alembic, Redis, Celery, pandas, Streamlit
운영·검증	Docker Compose, GitHub Actions, Railway, AWS EC2·RDS staging

검증된 현재 상태

832 passed 전체 회귀 실행 결과	96 passed ETL·검수·API 검증	Test #31 GitHub Actions 성공	Version 5 sale_price 규칙 반영
----------------------------------	-----------------------------------	--------------------------------------	--------------------------------------

핵심 성과

- FastAPI 와 PostgreSQL 로 검수 실행·저장, 이력 검색·상세 조회 API 를 구현하였습니다.
- 동일 CSV 는 SHA-256 파일 해시와 검수 버전으로 기존 결과를 재사용하고, partial unique index 로 동시 요청 중복을 방어하였습니다.
- Redis·Celery 비동기 검수와 sale_price 규칙을 연결하고, 합성 공급사 JSON 프로필 2 종을 공통 ETL 코드 변경 없이 처리하였습니다.

링크

GitHub	psy0635-ctrl/catalogguard-lite
Demo	CatalogGuard Lite Streamlit 앱

1. 문제 정의와 사용자 가치

기능 나열보다 상품 데이터가 실제로 왜 검수되어야 하는지를 먼저 정의했습니다.

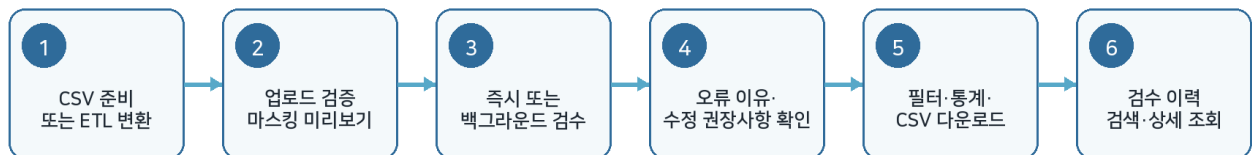
문제 정의

쇼핑몰 상품 카탈로그는 공급사와 운영 담당자마다 컬럼명과 입력 방식이 달라 같은 의미가 여러 형태로 저장될 수 있습니다. 이 상태가 누적되면 검색·필터·재고 관리·상품 노출의 정확도가 떨어지고, 사람이 CSV 를 반복 확인해야 하는 비용이 커집니다.

반복되는 문제	예시	서비스의 대응
형식 불일치	price 에 문자, 필수값 누락	업로드 검증과 규칙 기반 오류 탐지
표기 불일치	black·블랙·BLACK	색상·사이즈 별칭을 표준값으로 정규화
관계 오류	같은 그룹에 다른 카테고리	product_group_id 단위로 그룹 전체 비교
가격 관계 오류	sale_price 가 price 보다 큼	형식·0 이하·정상가 초과를 분리 검수
민감정보 혼입	설명에 이메일·전화번호	의심 정보 탐지와 미리보기 마스킹

사용자 흐름

사용자 흐름



사용자가 받는 결과

검수 상태, 오류 항목, 상품 ID, 오류 이유, 수정 권장사항, 위험 수준을 한글로 제공하며 필터 적용 결과를 CSV 로 내려받을 수 있습니다.

MVP 범위

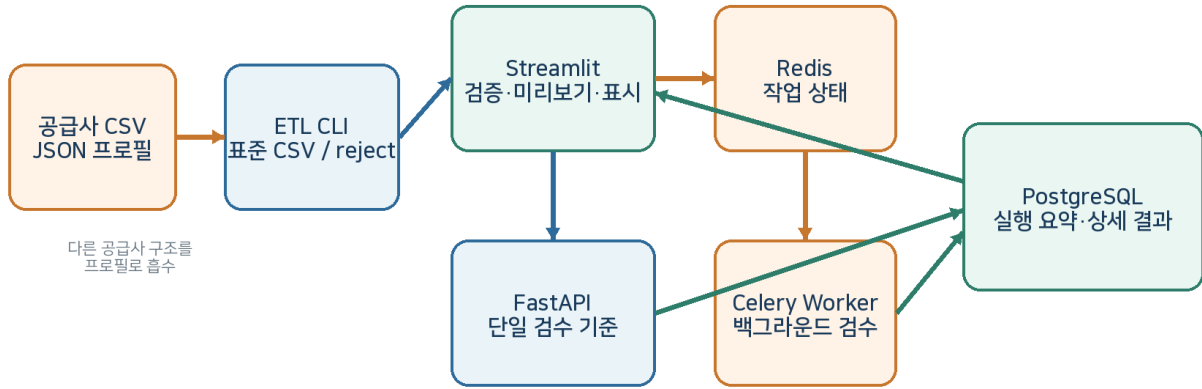
- CSV 업로드와 합성 공급사 프로필 2 종 변환부터 검수·통계·다운로드·이력 조회까지 한 흐름으로 연결하였습니다.
- AI 모델을 핵심 기능처럼 과장하지 않고, 현재 구현의 중심을 규칙 기반 데이터 품질 검수로 명확히 구분하였습니다.

2. 시스템 구조와 책임 분리

화면, API, 작업 큐, 데이터베이스, ETL의 역할을 나누어 같은 기능이 중복되지 않도록 하였습니다.

CatalogGuard Lite 전체 구조

공급사 변환, 동기·비동기 검수, 저장과 화면 표시의 책임을 분리했습니다.



계층별 역할

구성	주요 책임	설계 판단
Streamlit	파일 사전 검증, 마스킹 미리보기, 결과·통계·다운로드	전체 검수 규칙을 다시 실행하지 않음
FastAPI	동기·비동기 요청, 검수 단일 기준, 이력 API	화면과 저장 결과의 기준을 서버로 통일
PostgreSQL	실행 요약과 상세 결과, 검색, 중복 제약	브라우저 재시작과 동시 요청에도 결과 유지
Redis·Celery	작업 상태와 백그라운드 실행	긴 검수를 HTTP 요청 수명과 분리
ETL CLI	합성 공급사 프로필 2종 컬럼 매핑, 정상·reject·요약 파일 생성	공급사별 코드를 만들지 않고 JSON 프로필로 구조 차이를 흡수

핵심 기술 판단

- **단일 검수 기준** — 서버 검수 결과를 Streamlit 화면·통계·필터·다운로드에 재사용하여 이중 검수를 제거하였습니다.
- **원본 보존** — 업로드 미리보기는 원본 DataFrame의 깊은 복사본에만 마스킹하여 검수 정확도와 화면 보안을 분리하였습니다.
- **응답 방어** — 서버 상세 응답의 실행 ID, created, 요약 수치, 상세 필드를 검증한 뒤에만 화면 상태에 저장합니다.

3. 핵심 검수 기능

등록된 검수 규칙을 공통 ValidationIssue 형태로 통일하여 API 와 화면에 같은 결과를 전달합니다.

검수 범위

분류	검수 내용	대표 예시
입력·형식	필수값, 카테고리, 재고, price·sale_price 형식	price="가격문의" → 가격 오류
중복	상품 ID, 상품명 후보, 완전 중복	같은 속성이 모두 같은 상품
패션 옵션	색상·사이즈 비표준, 옵션 조합 중복	블랙 → BLACK, medium → M
그룹 관계	같은 그룹의 카테고리 불일치	TOP 과 BOTTOM 이 같은 그룹에 존재
가격	0 이하, 정상가·할인가 관계, 카테고리 이상치	price 50,000 / sale_price 60,000
내용·보안	상품명-카테고리, 금지어, 개인정보 의심	설명에 이메일·전화번호 포함

패션 데이터 정규화 예시

BLACK black / 블랙 / BLACK	M medium / M	XXL 2XL / XXL	FREE 프리사이즈 / FREE
------------------------------------	------------------------	-------------------------	-----------------------------

그룹 단위 판단

개별 행만 검사하면 같은 상품 그룹 안에서 발생하는 관계 오류를 찾기 어렵습니다. 다수결로 임의의 정답 카테고리를 만들지 않고, 불일치가 존재한다는 사실을 참여 상품 전체에 연결했습니다. 결과는 사용자가 업로드한 원본 행 순서를 유지합니다.

결과 계약

모든 규칙은 severity, product_id, product_group_id, message 를 가진 ValidationIssue 로 통일합니다. 화면에서는 검수 상태·오류 항목·오류 이유·수정 권장사항·위험 수준으로 변환합니다.

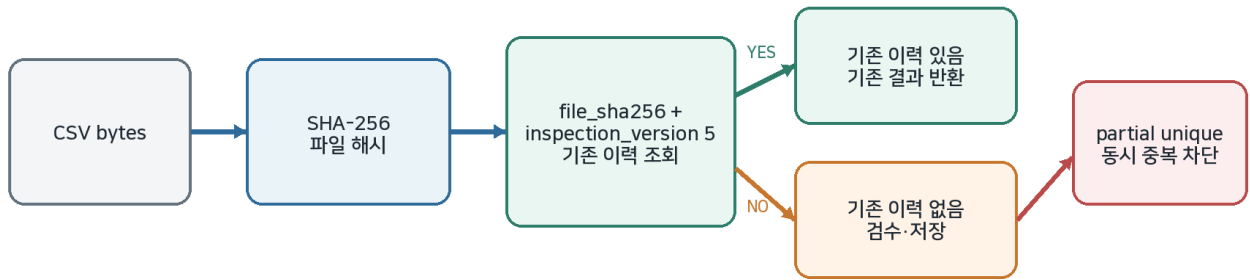
안전한 결과 표시

- 내부 상세 값은 JSON 구조로 저장해 작은따옴표와 쉼표가 포함된 값도 안전하게 전달합니다.
- 손상되거나 예상하지 못한 JSON 은 사용자 화면 전체를 중단시키지 않고 기본 안내 문구로 처리합니다.
- 다운로드 CSV 는 Excel 수식 삽입 위험이 있는 =, +, -, @ 시작 값을 복사본에서만 안전하게 처리합니다.

4. 동일 CSV 결과 재사용과 검수 버전 관리

불필요한 재검수를 줄이되 규칙 변경 후에는 같은 파일도 새 기준으로 다시 검사하도록 설계했습니다.

동일 CSV 결과 재사용과 동시 요청 방어



문제와 해결

문제	해결 방법	효과
같은 파일을 반복 업로드	CSV bytes 의 SHA-256 해시 생성	파일명과 무관하게 내용이 같은 파일 식별
새 규칙인데 과거 결과 재사용	file_sha256 + inspection_version 으로 조회	규칙 버전이 바뀌면 같은 CSV 도 재검수
동시에 같은 요청이 들어옴	PostgreSQL partial unique index	애플리케이션 조회 이후 경쟁 요청도 DB 가 차단
화면 세션 내 반복 제출	saved hash 와 활성 job_id 확인	불필요한 POST·GET 호출 감소

inspection_version 5 로 올린 이유

sale_price 형식·0 이하 값·정상가보다 큰 관계를 검사하는 규칙이 추가되었습니다. 파일 해시만 사용하거나 버전 4 를 유지하면 과거 결과 재사용되어 새로운 할인 가격 검수가 실행되지 않을 수 있으므로 버전을 5 로 변경하였습니다.

DB migration 이 없었던 이유

sale_price 원본 상품 전체를 별도 테이블에 저장하지 않고 검수 실행과 결과만 저장하므로 기존 DB 컬럼 구조는 바뀌지 않았습니다. 규칙 버전만 변경하고 Alembic migration 은 추가하지 않았습니다.

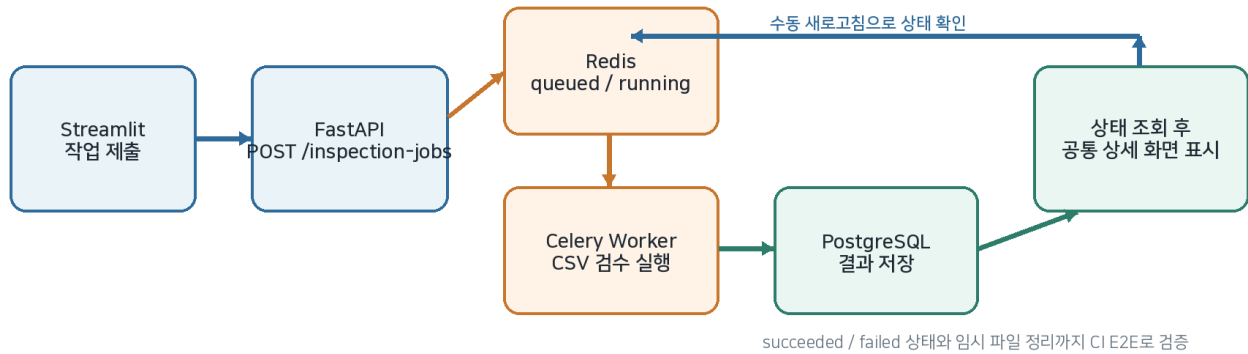
면접에서 설명할 핵심

- 중복 저장을 막는 것과 불필요한 검수 실행을 줄이는 것은 다른 문제이므로 API 사전 조회와 DB 제약을 함께 사용했습니다.
- 버전 값은 기능 소개용 숫자가 아니라 검수 결과 재사용의 정확성을 보장하는 식별자입니다.

5. 비동기 처리와 자동 검증

긴 검수 작업을 요청 수명에서 분리하고 실제 서비스들을 연결한 흐름을 GitHub Actions 에서 확인했습니다.

Redis·Celery 비동기 검수 흐름



비동기 기능

항목	구현 내용
작업 상태	queued, running, succeeded, failed
제출·조회	작업 제출 API와 명시적 상태 조회 API
결과 표시	완료 후 동기 검수와 같은 상세 결과 렌더러 사용
정리	성공·실패 후 임시 CSV 파일 정리
중복 방지	같은 세션의 활성 job_id와 파일 해시로 반복 제출 방지

GitHub Actions Test workflow

순서	자동 검증 범위
1	PostgreSQL 18·Redis 7.4 서비스 컨테이너 시작
2	Alembic upgrade head와 E2E 제외 전체 pytest 실행
3	Celery Worker와 FastAPI 프로세스 시작
4	/health·/ready, 비동기 제출, polling, 결과 저장, 동일 파일 재사용, 임시 파일 정리
5	Streamlit 시작 후 /_stcore/health HTTP 200·ok 확인

배포·운영 검증 범위

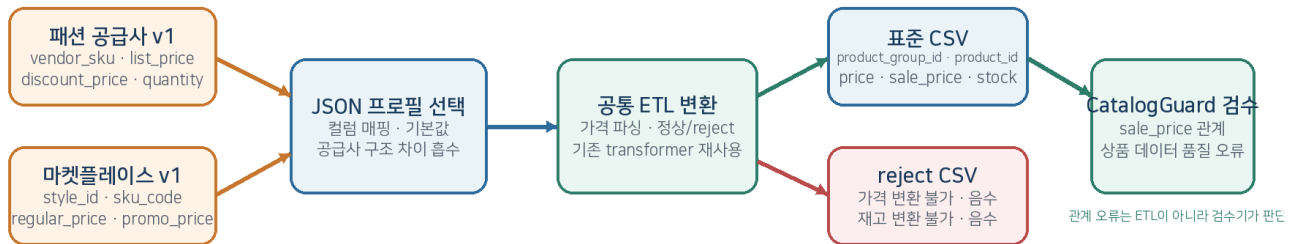
- **Railway** — Railway에서 FastAPI /health·/ready와 구조화 요청 로그를 확인하였습니다.
- **AWS** — AWS EC2·RDS staging에서 Docker, Alembic, TLS DB 연결, FastAPI와 별도 Streamlit 흐름을 검증한 뒤 비용 절감을 위해 리소스를 중지하였습니다.
- **범위 제한** — 운영 환경 Redis·Celery 배포는 아직 검증하지 않았으며, 현재 비동기 근거는 로컬·CI E2E입니다.

6. 공급사 프로파일 2 종 ETL 과 할인 가격 검수

서로 다른 합성 공급사 CSV 구조를 JSON 프로파일로 표준화하고, 변환 오류와 상품 품질 오류의 책임을 분리했습니다.

공급사 프로파일 2종과 공통 ETL 흐름

공급사별 코드를 추가하지 않고 JSON 매핑 프로파일만 교체하여 같은 변환기를 재사용합니다.



합성 공급사 프로파일 비교

프로파일	원본 → 표준 컬럼	설계 의미
패션 공급사 v1	vendor_sku → product_group_id, product_id	그룹 ID와 개별 ID를 한 키로 처리
패션 공급사 v1	discount_price → sale_price	선택 할인 가격 컬럼 처리
마켓플레이스 v1	style_id → product_group_id	상품 그룹 키를 별도로 유지
마켓플레이스 v1	sku_code → product_id	같은 그룹 아래 개별 SKU 유지
마켓플레이스 v1	promo_price → sale_price	할인 가격 관계 검수로 전달
공통 ETL	list/regular_price → price · 재고 → stock	공통 파서와 기본값을 재사용

ETL 과 CatalogGuard 검수기의 책임

ETL 에서 reject	표준 CSV 에 남겨 검수기가 판단
필수 원본값 누락, 가격 변환 실패·음수, 재고 변환 실패·음수	중복 상품 ID, 비표준 색상·사이즈, 가격 이상치, sale_price > price

sale_price 정책

선택 컬럼 컬럼 없음·빈 값 정상	≤ price 같거나 낮으면 정상	> price 관계 오류	≤ 0 / 형식 오류 별도 가격 오류
-----------------------	-----------------------	------------------	-------------------------

안전성과 검증

- 입력·출력 경로 충돌을 거부하고, 임시 파일 작성 후 원자적으로 교체하여 실패 시 기존 파일을 보호합니다.
- 두 번째 샘플 CLI 는 3 행 중 정상 변환 2 행, reject 1 행, 종료 코드 0 을 확인하였습니다.
- 지정 ETL 테스트는 27 passed, ETL·검수 서비스·API 관련 검증은 96 passed 입니다.

현재 제한

합성 공급사 프로파일은 2 종이며 사용자가 수동으로 선택합니다. 실제 외부 공급사 연동, 웹 수집, DB 직접 적재, 대용량 streaming, 증분 ETL, 자동 공급사 감지는 아직 지원하지 않습니다.

7. 검증 결과, 기술 판단과 한계

숫자를 과장하지 않고 어떤 환경에서 무엇을 검증했는지와 적용하지 않은 선택까지 함께 기록했습니다.

검증 결과

검증 항목	결과	해석
로컬 전체 pytest	832 passed, 26 skipped, 2 deselected	기본 설정에서 performance-e2e 제외, 서비스 조건에 따라 일부 skip
ETL·검수·API 관련	96 passed (지정 ETL 27 passed)	프로필 2 중 변환·reject·sale_price·API 계약 호환성
GitHub Actions	Test #31 success	CI 858 passed·비동기 E2E 1 passed·Streamlit HTTP 200
Streamlit 서버	/_stcore/health 200, ok	서버 시작 스모크이며 브라우저 전체 기능 검증은 아님

동기 검수 벤치마크

행 수	1 회 중앙값	연속 2 회	Python peak
100	0.009544 초	0.018815 초	0.177 MiB
1,000	0.074266 초	0.150817 초	1.603 MiB
5,000	0.371740 초	0.867245 초	6.485 MiB
10,000	0.875495 초	1.645721 초	12.939 MiB

개발 PC 합성 데이터 측정이며 운영 트래픽 성능을 보증하지 않습니다. 같은 검수를 두 번 실행했을 때 약 1.88~2.33 배가 소요된 결과를 근거로 Streamlit·FastAPI 이중 검수 제거를 우선했습니다.

PostgreSQL 성능 판단

- 10,000 개 실행·100,000 개 상세 결과 합성 데이터에서 병목으로 판단할 쿼리를 발견하지 않아 새 인덱스를 추가하지 않았습니다.
- 목록 복합 인덱스는 중앙값 차이가 0.022ms 에 불과했고, 상세 복합 인덱스는 더 느리고 더 커서 기각했습니다.
- 상세 API 는 결과 수와 무관하게 SELECT 2 회로 끝나 N+1 이 발생하지 않음을 테스트로 고정했습니다.

현재 한계

구분	현재 상태
운영 비동기	Redis·Celery 운영 배포 미검증
작업 제어	취소·세부 진행률 미지원
데이터 유입	합성 공급사 프로필 2 중, 수동 선택, CSV 만 지원
대용량 처리	streaming·증분 ETL·동시 트래픽 측정 미지원

30 초 소개

면접 답변

CatalogGuard Lite 는 서로 다른 공급사 상품 CSV 를 JSON 프로필로 표준화하고 누락·중복·가격·카테고리·개인정보 문제를 검수하는 데이터 품질 서비스입니다. FastAPI 와 PostgreSQL 로 결과 이력을 관리하고, 동일 CSV 는 파일 해시와 검수 버전으로 기존 결과를 재사용하도록 설계했습니다. 합성 공급사 프로필 2 종을 공통 ETL 코드 변경 없이 처리했으며, Redis·Celery 비동기 흐름을 GitHub Actions E2E 로 검증했습니다.

최신 기준: commit e5e2ffa · inspection_version 5 · GitHub Actions Test #31 success